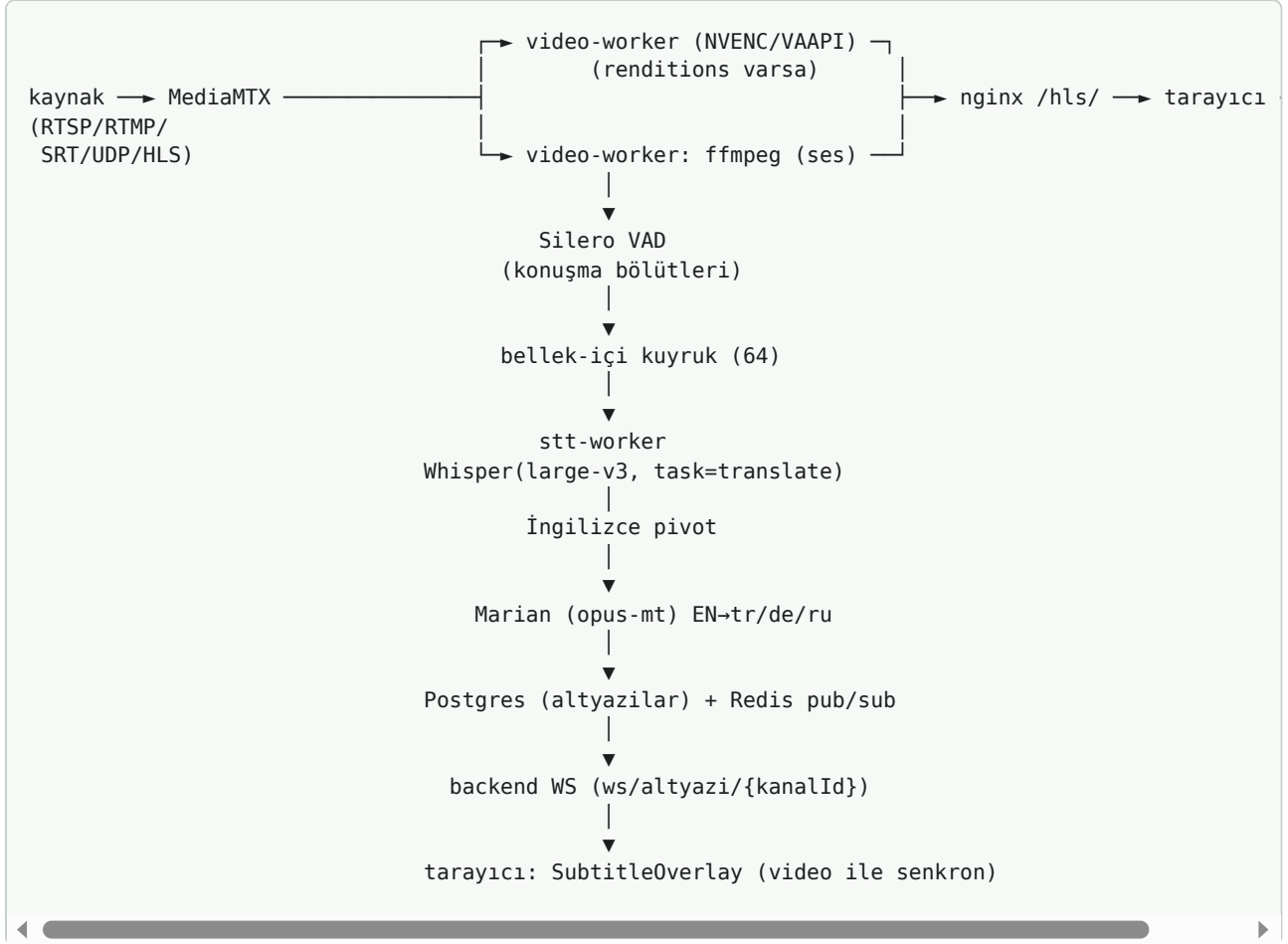


Kaynaktan ekrana

Bölüm A: bir kaynak yayının video ve altyazı olarak izleyiciye ulaşana kadar geçtiği tüm mimari.

Bölüm B: gecikmeyi önlemek için ne yapıyoruz, nasıl yapıyoruz, ve henüz yapılmayan ama yapılması gereken.

A • 1. Tek bakışta



Kritik nokta: video yolu ve altyazı yolu birbirinden tamamen bağımsız iki bağlantı. Aralarında doğrudan hiçbir çağrı yok — senkron olup olmadığı yalnızca tarayıcıda, iki farklı zaman damgası karşılaştırılarak anlaşılıyor (bkz. A.5).

A • 2. Video yolu

Bugün sağlıklı çalışıyor, bu oturumun odağı değil ama tam resim için:

- 1 MediaMTX kanalın `sourceUrl` 'ünü çeker. Protokol (RTSP/RTMP/SRT/UDP/HLS) ayrı bir alan değil — URL'nin şemasından (`rtsp://`, `https://...m3u8`) çıkarılıyor.

- 2 Kanalda `renditions` tanımlıysa video-worker ffmpeg ile ek kaliteler üretir. Kodlayıcı `CHANNELS_ENCODER`'a göre seçilir: `NVENC` (NVIDIA), `VAAPI` (Intel/AMD), `YAZILIM` (libx264, CPU). Sonuç MediaMTX'e geri yazılır.
- 3 MediaMTX HLS playlist'i (`.m3u8`) bellekte üretiyor, segmentleri (`.mp4`) diske yazıyor.
- 4 Tarayıcı, frontend'in nginx'i üzerinden `/hls/<kanal>/index.m3u8`'i çekip hls.js ile oynatıyor.

A · 3. Altyazı yolu – adım adım

Videodan tamamen ayrı bir bağlantı: video-worker, MediaMTX'ten sesi de ayrıca RTSP ile kendi çekiyor (`MEDIAMTX_RTSP_URL=rtsp://mediamtx:8554`).

- 1 ffmpeg, `rtsp://mediamtx:8554/<path>`'ten sesi 16 kHz mono s16le olarak çekiyor — Silero VAD ve Whisper'ın ikisi de bu formatı istiyor.
- 2 Silero VAD (ONNX, v5) konuşma/sessizlik ayrımı yapıyor. `VAD_MIN_SILENCE_MS=400` sessizlik bölütü kapatıyor; `VAD_MAX_SEGMENT_MS=4000`'ı aşan konuşma zorla kesiliyor (`kesik=true`); `VAD_OVERLAP_MS=800` bir sonraki bölütle örtüşerek bağlamı taşımaya çalışıyor. `VAD_MODEL_VERSION` kozmetik bir alan değil — yanlış sürüm ONNX'i patlatır ya da sessizce boş altyazı üretir.
- 3 Bölüt, `VadService`'in kanal başına açtığı bellek-içi kuyruğa giriyor — `ArrayBlockingQueue<64>`, sabit, `.env`'den ayarlanamıyor. Kuyruk dolunca `offer()` yeni bölütü atıyor, `take()` hep en eskiyi alıyor — kuyruk bir kez dolduğunda içi bayat verilerle kalıyor (bkz. B-1).
- 4 "stt-gonderici" iş parçacıkları kuyruktan alıp `POST /transcribe?channel=&start=&end=` ile ham PCM'i stt-worker'a gönderiyor.
- 5 Whisper (`large-v3`, `task=translate`) sesi doğrudan İngilizce pivota çeviriyor, aynı zamanda kaynak dili tahmin ediyor. Eşzamanlılık `Semaphore(STT_MAX_CONCURRENCY)` ile sınırlı.
- 6 Marian (Opus-MT) İngilizce pivottan `tr/de/ru`'ya çeviriyor. Bu oturumda GPU'ya taşındı, dil başına ayrı kilit kondu — bkz. B-2.
- 7 Sonuç video-worker'a dönüyor, iki yere yazılıyor: **Postgres** (`altyazilar` tablosu, mutlak `baslangic/bitis` zaman damgalarıyla — doğruluk kaynağı burası) ve **Redis** (`altyazi:<kanalId>` 'e pub/sub — yalnızca bildirim, kaybolursa yoklama toparlar).
- 8 Backend'in `SubtitleBroadcaster`'ı bu Redis kanalını dinliyor, izleyen `ws/altyazi/{channelId}` oturumlarına anında iletiyor. Bu uç kimlik doğrulaması istemiyor.
- 9 Tarayıcıda `SubtitleOverlay`, geçmiş için `GET /api/channels/{id}/altyazilar`'ı bir kez okuyor, canlı için WebSocket'e bağlanıyor. Her 250ms'de bir o anki oynatma zamanını önbellekteki bölütlerin (`baslangic, bitis`) aralığıyla karşılaştırıp eşleşeni ekrana basıyor. İzleyici karodaki "Altyazı" menüsünden bir dil seçmeden bu hiç başlamıyor — varsayılan `kapa1ı`, kalıcı değil.

A · 4. .env değişkenleri – tam referans

Donanım / kodlayıcı

<code>CHANNELS_ENCODER</code>	Kanal rendition kodlayıcısı: NVENC VAAPI YAZILIM
<code>VIDEOS_ENCODER</code>	Yüklenen video işleme kodlayıcısı
<code>CONTAINER_RUNTIME</code>	Docker runtime: nvidia ya da runc
<code>NVIDIA_VISIBLE_DEVICES</code> / <code>NVIDIA_DRIVER_CAPABILITIES</code>	Konteynere açılan GPU özellikleri
<code>MEDIA_DEVICE</code> / <code>WORKER_MEDIA_DEVICE</code>	<code>/dev/dri</code> (VAAPI) geçişi; NVIDIA'da zararsız yer tutucu
<code>CHANNELS_MAX_ACTIVE</code>	Aktif kanal üst sınırı (bu oturumda compose'a kablolandı)

VAD

<code>VAD_ENABLED</code>	VAD hattının açık/kapalı olması
<code>VAD_MODEL_PATH</code>	Silero VAD ONNX modelinin imaj içi yolu
<code>VAD_MODEL_VERSION</code>	v4 v5 — modelin beklediği örnek/bağlam biçimi
<code>VAD_MAX_CHANNELS</code>	Aynı anda VAD çalıştırılacak kanal üst sınırı
<code>VAD_SEGMENT_DIR</code>	Bölütlerin diske yazılacağı yol
<code>VAD_MAX_SEGMENT_MS</code>	Zorla kesmeden önce azami bölüt süresi
<code>VAD_MIN_SILENCE_MS</code>	Bölütü kapatan sessizlik süresi
<code>VAD_MIN_EMIT_MS</code>	Kısa bölüt birleştirme penceresi — şu an 0, etkisiz
<code>VAD_OVERLAP_MS</code>	Zorla kesimde bırakılan örtüşme

STT (çözümleme + çeviri)

<code>STT_MODEL</code>	Whisper model boyutu: tiny...large-v3
<code>STT_DEVICE</code>	cpu cuda — Whisper ve (bu oturumdan sonra) Marian için
<code>STT_COMPUTE_TYPE</code>	float16 int8_float16 int8
<code>STT_BEAM_SIZE</code>	Whisper arama genişliği
<code>STT_BATCH_SIZE</code>	Tek sesin kendi içinde yığılması
<code>STT_MAX_CONCURRENCY</code>	Aynı anda GPU'da çözümlenebilecek bölüt sayısı
<code>STT_TARGET_LANGS</code>	Hedef diller: tr,de,ru
<code>STT_URL</code>	video-worker'ın POST ettiği adres

VAD_STT_ENABLED	Bölütlerin STT'ye gönderilip gönderilmeyeceği
STT_RUNTIME	stt-worker konteynerinin docker runtime'ı

Altyazı gecikme/kapsama ölçümü

ALTYAZI_HLS_GERIDE	İzleyiciyi canlı kenardan kaç segment geriye alır — bütçeyi büyütüp gecikmeyi saklar, çözmez
ALTYAZI_BUTCE_MS	Yanılıcı isim — hiçbir şeyi düşürmüyor, yalnızca kapsama yüzdesini hesaplamak için
ALTYAZI_RAPOR_ARALIGI	SubtitleLagMetrics'in rapor sıklığı (varsayılan 60s)

A · 5. Kritik davranış – mutlak zaman damgası

`SubtitleOverlay.tsx`'teki eşleştirme mutlak zaman üzerinden çalışıyor: `baslangic <= playingDate() < bitis`.

EN SİNSİ DAVRANIŞ

Bir bölütün üretimi ne kadar sürerse sürsün `bitis` sabit kalır. Üretim 90 saniye sürdüyse, oynatma zamanı o pencereyi çoktan geçmiş olur. Hiçbir hata çıkmaz, loglar temiz görünür, ekran sessizce boş kalır — bölüm sonsuza dek "görünmedi" sayılır. Bu yüzden "altıyazı akıyor" teşhisinde önce her zaman gecikmeye bakılmalı; kodda hata aranırsa bulunamaz, çünkü hata yok, zamanlama var.

B · 1. Şu an sistemde duran önlemler

Bunların hiçbirisi kök sebebi çözmiyor; ya semptomu saklıyor ya da durumu kötüleşmeyi engelliyor.

- ✓ **ALTYAZI_HLS_GERIDE** — izleyiciyi canlı kenardan geriye alıyor (`liveSyncDurationCount`).
Bütçe = bu sayı × segment süresi.
Çözüm değil, saklama: gerçek gecikme (90 sn) hâlâ orada, sadece izleyicinin fark etme eşiği büyütülüyor.
- ✓ **STT_MAX_CONCURRENCY** (Semaphore) — GPU'da aynı anda kaç bölüt çözümlenebileceğini sınırlıyor, bellek taşmasını önüyor.
Ama bu sınır aynı zamanda darboğazın kendisi: 16 kanal, 4 kişilik bir hatta sıraya giriyor.
- ✓ **VAD_MAX_SEGMENT_MS (4000ms)** — bölütleri kısa tutarak bekleme süresini sınırlıyor. Kısa bölüt = daha az bekleme, ama daha az bağlam.
- ✓ **STT_COMPUTE_TYPE=int8_float16** — daha az hassasiyetle ~%30 hız kazanıyor.
Kalite etkisi hâlâ ölçülmedi.
- ✓ **Sınırlı kuyruk (64)** — bellekte sınırsız birikmeyi önüyor.
Ama yanlış taraftan düşürüyor: yeni bölüt reddediliyor, en eski hep işleniyor — kuyruk dolunca içi kalıcı olarak bayat kalıyor.

B · 2. Bu oturumda yapılan gerçek düzeltme

`stt-worker/app/translate.py` — çeviri artık GPU'da ve dil başına ayrı kilitle.

Öncesinde üç dilin (TR/DE/RU) çevirisi tek bir kilitle, CPU'da, sistem genelinde tek sıraya giriyordu. Ölçülen tavan **4,39×** gerçek zamandı; Whisper'ı bedavaya indirseniz bile bu sayı değişmiyordu, çünkü çeviri ondan sonra, ayrı bir seri adımdı.

```
- self._lock = threading.Lock()                # tek, tüm diller ortak
+ self._locks: dict[str, threading.Lock] = {}    # dil başına bir tane

# load() içinde, her dil için:
+ model.to(SETTINGS.device)                    # artık cuda'ya taşınıyor
+ self._locks[language] = threading.Lock()

# _translate_one içinde:
- with self._lock, torch.no_grad():
+ with self._locks[language], torch.no_grad():
+     batch = batch.to(SETTINGS.device)
```

Sonuç: TR çevirisi artık DE'yi, bir kanalın çevirisi başka bir kanalını bloklamıyor. Aynı dilin ardışık çağrıları hâlâ seri (aynı model nesnesine eşzamanlı `generate()` çağrısı güvenli sayılmıyor) — bu bilerek korundu.

● **test edilmedi** `stt-worker` imajı henüz yeniden kurulmadı, gerçek GPU üzerinde ölçüm yapılmadı. Sözdizimi kontrolü (`py_compile`) geçti.

B · 3. Yapılması önerilen – henüz yapılmadı

Öncelik sırasıyla, getiri/efor oranına göre.

- 1 Yalnızca izlenen kanalı çözümle** — `VadService.java:134`, izleyici koşulu yok. Aktif her kanal işleniyor, kimse izlemese de.
Kazanç: gereken kapasite = kanal sayısı → izlenen kanal sayısı (genelde 1). 16 kanallı kurulumda potansiyel **16 kat**. Altyapı hazır: `SubtitleBroadcaster.sessions` zaten izleyici sayısını tutuyor.
- 2 Kuyruğu doğru taraftan düşür** — en eskiyi düşür, ya da kuyruğa girmeden önce segmentin yaşına bakıp bütçeyi çoktan aşmış olanı hiç göndermeden atla.
Şu an hiçbir yaş kontrolü yok — 90 saniyelik bayat bir segment de tam öncelikle GPU'ya gönderiliyor.
- 3 `STT_MAX_CONCURRENCY`'yi ölçerek yükselt** — tek kanalda GPU bazen %0 kullanımda ölçüldü. VRAM boşluğu varsa (`large-v3` ~1,6 GB + 3 dil Marian ~0,9 GB) bu sayı `nvidia-smi` ile izlenerek yükseltilebilir.
- 4 Kanallar arası yığınlama (batching)** — farklı kanallardan gelen bölütleri aynı ileri geçişte birleştirmek henüz yok.
`docs/olcekleme-planı.md`'de "doğrulanmamış varsayım" olarak işaretli.

B · 4. Özet tablo

Her önlemin gecikmeye etkisi ve durumu.

ÖNLEM	NE YAPIYOR	KÖK SEBEBİ ÇÖZÜYOR MU
<code>ALTYAZI_HLS_GERIDE</code>	İzleyiciyi geriye alır	Hayır — saklar
<code>STT_MAX_CONCURRENCY</code>	GPU'yu korur	Hayır — darboğazın kendisi
Sınırlı kuyruk (64)	Bellek taşmasını önler	Hayır — yanlış taraftan düşürür
Çeviri → GPU + dil kilidi	Seri çeviriyi kaldırır	Kısmen — test edilmedi
İzleyici filtresi (önerilen)	Gereksiz kanalı hiç işlemez	Evet — en büyük kazanç
Yaşa göre düşürme (önerilen)	Bayat segmenti hiç göndermez	Evet — israfı keser

UNUTULMAMASI GEREKEN

Hiçbiri, altyazının izleyiciye ulaşma anını tarayıcıdaki mutlak zaman damgası kontrolünden (A-5) muaf tutmuyor: bölüt ne kadar hızlı üretilirse üretilsin, oynatma zamanı `bitis`'i geçtiyse o bölüt asla ekrana çıkmaz. Hedef her zaman "bütçenin (8 sn) içinde üretmek", "üretilince göstermek" değil.